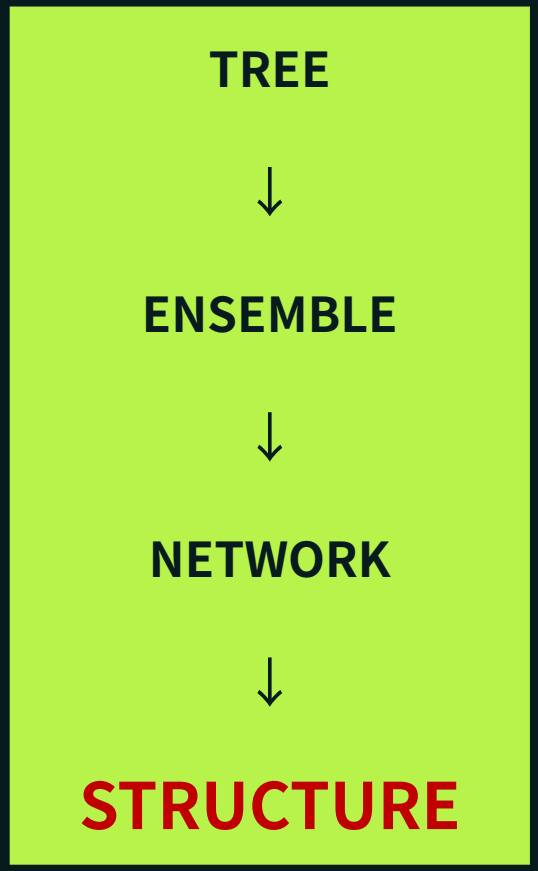
Trees, Neural Networks and Unsupervised Learning

Weeks 7–9 · Summary and Cheat Sheet



Financial Machine Learning · QRM3007

Contents

01	**Trees and Ensembles**	**Week 7** Gini · bagging · boosting · importance · iForest
02	**Neural Networks**	**Week 8** Neuron · XOR · gradient descent · backprop · AE
03	**Unsupervised Learning**	**Week 9** K-Means · PCA · Marchenko–Pastur denoising
04	**Cheat Sheet**	**Weeks 7–9** Formulas · dials · detectors · tool picker

One flexible learner is fragile — a diverse, disciplined team is not

01

Trees and Ensembles

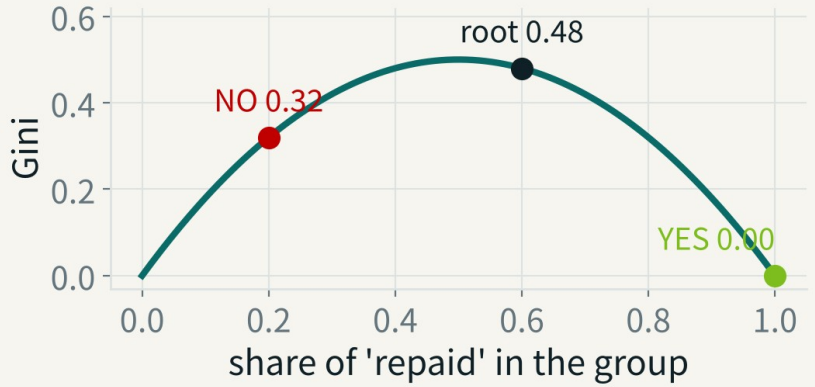
Week 7 · one tree, a forest, a relay team, and the isolation detector

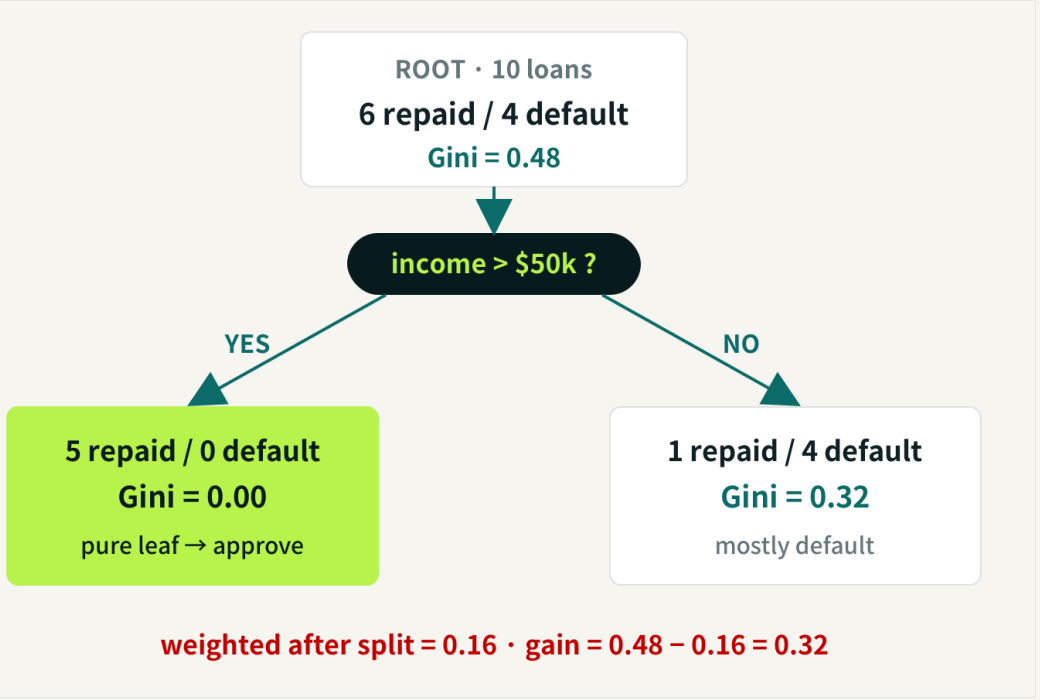
A tree picks the split that buys the most purity

Ten loans, 6 repaid / 4 default — is "income > $50k?" a good question?

Gini impurity — 0 means pure

$$G = 1 - \sum_k p_k^2$$

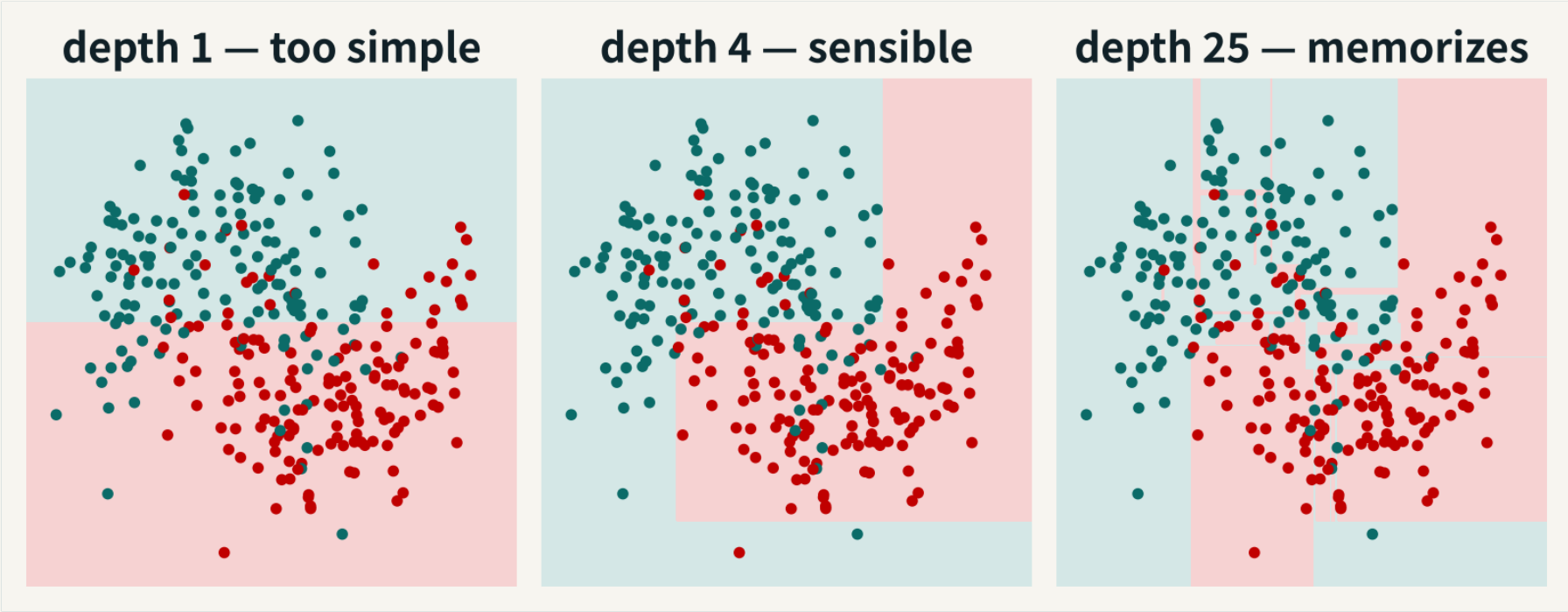




Gini drops 0.48 → 0.16 after the question; the tree keeps the split with the biggest drop.

Deeper trees draw finer boundaries — until they memorize

The same 300 noisy points, split by trees of depth 1, 4 and 25



Depth 25 carves an island around every stray point — perfect in-sample, fragile out.

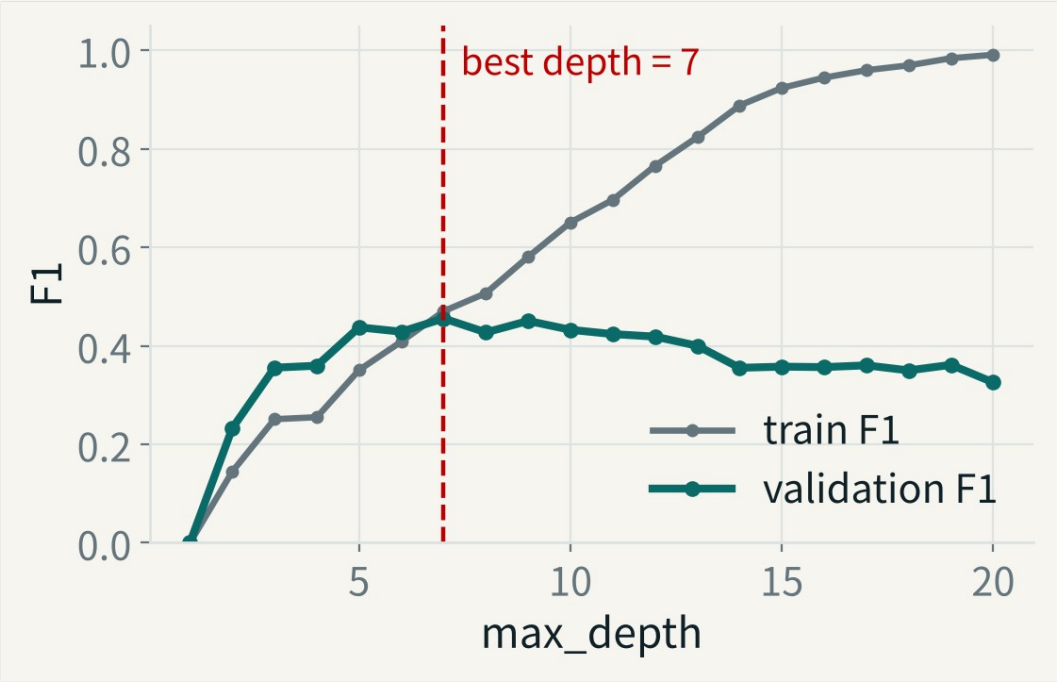
Depth is the tree's complexity dial

Train vs validation F1 by max_depth — 20,000 synthetic rows, 1% positive

TAMING KNOBS

- max_depth: number of questions
- min_samples_leaf: no tiny leaves

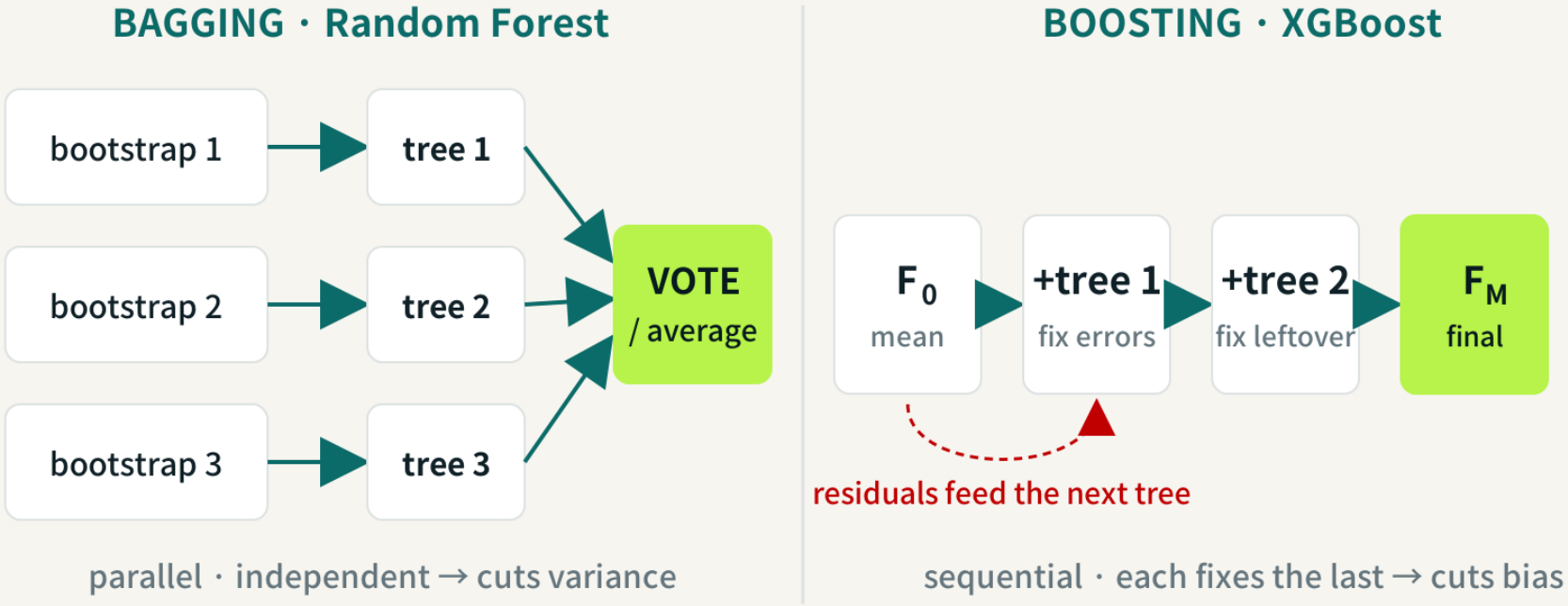
Validation F1 peaks at depth 7



Train F1 keeps rising with depth; validation turns down — even a tamed tree stays shaky.

A forest votes in parallel; boosting corrects in sequence

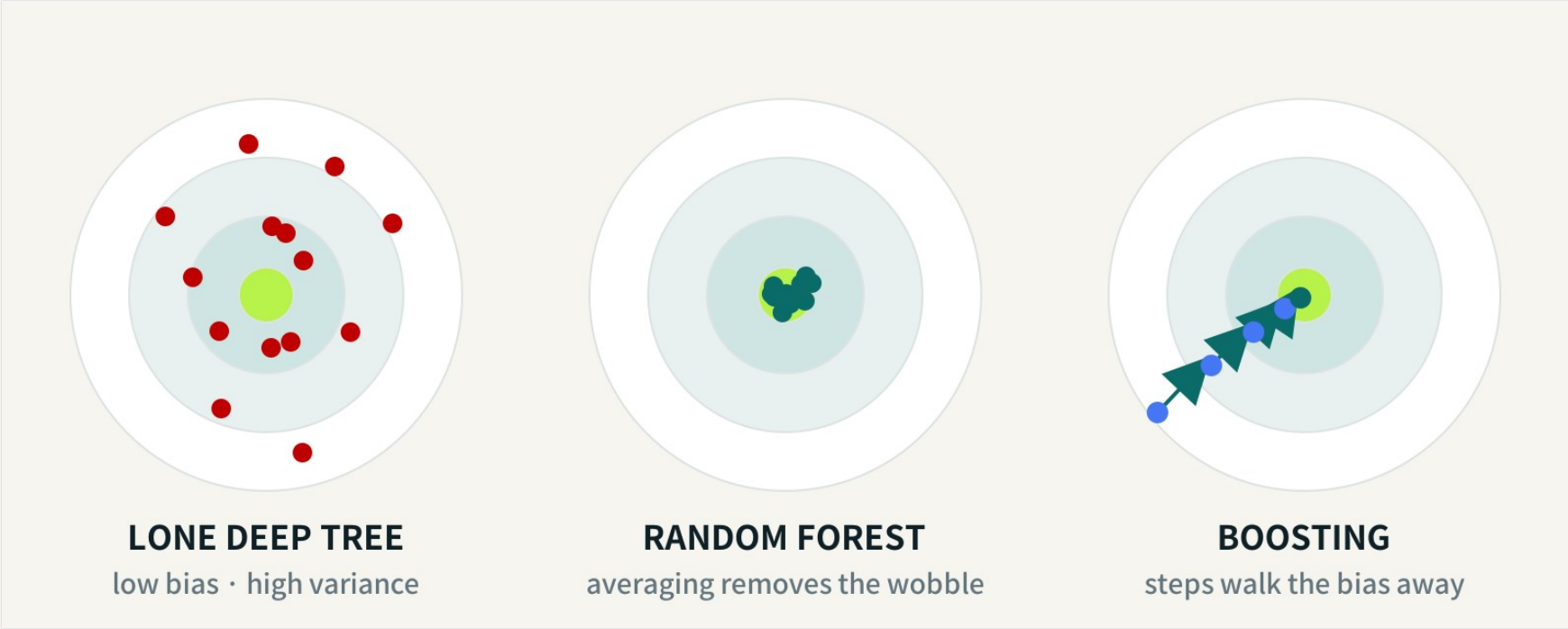
Bagging = bootstrap rows + random feature subsets · boosting = small steps on residuals



Committee vs relay team — both beat a lone tree.

A forest removes wobble; boosting removes the systematic miss

Dots = predictions from models trained on different samples; bullseye = the truth



Bagging targets variance, boosting targets bias — both beat a lone deep tree.

Averaging cancels errors only when the trees disagree

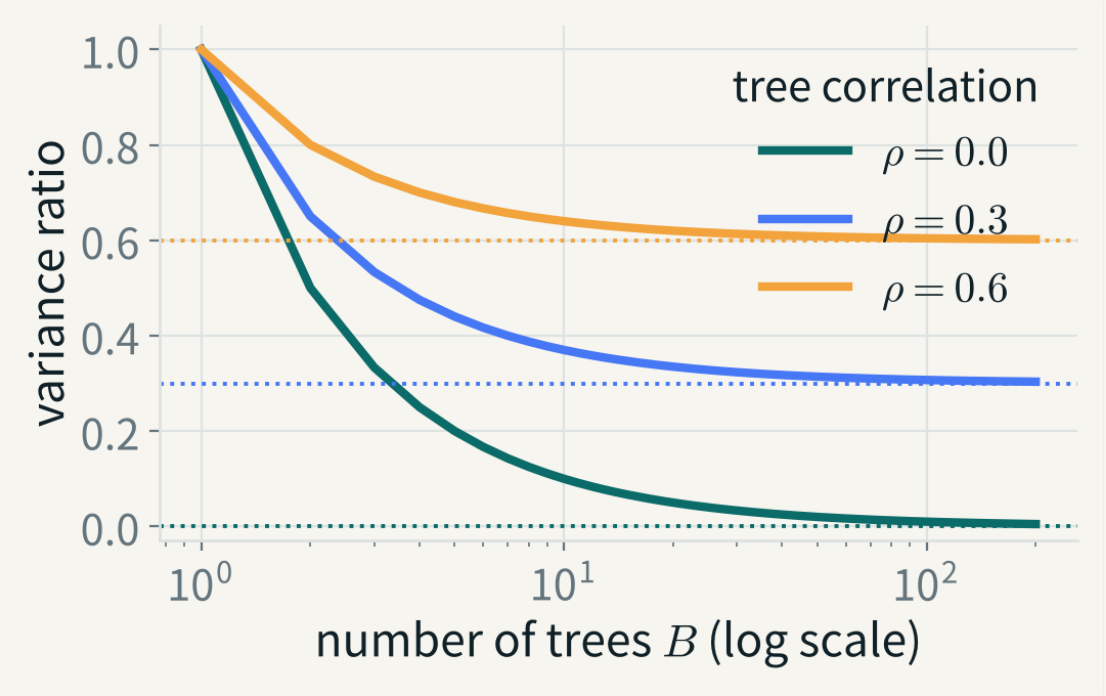
Variance of an average of B trees, relative to one tree, for three levels of tree correlation

Variance of the forest average

$$\mathrm{Var}\left[\hat{f}_{\mathrm{bag}}\right] = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2$$

Correlation sets a floor

Random rows + random features lower it



Adding trees only shrinks the second term — diversity is what really pays.

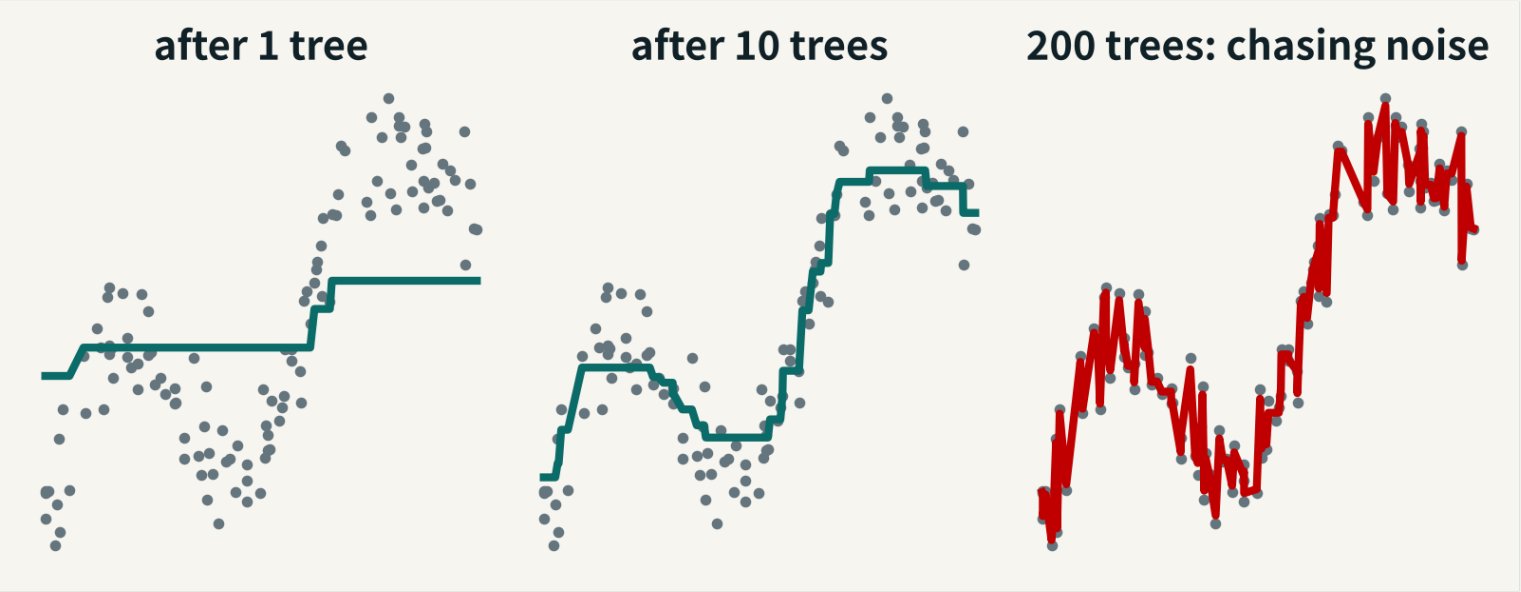
Boosting adds small corrections on the leftover errors

Gradient boosting with depth-2 trees and step 0.3 on a noisy curve

Boosting update

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$$

Old model
plus a small step
toward the leftovers



Each tree fixes part of what is left; in the toy example the error melts 1.00 → 0.25 → 0.06.

Too many trees chase noise — tune the learning rate and tree count on validation.

Bagging cuts variance; boosting cuts bias

	Random Forest (bagging)	Boosting (XGBoost)
How trees are built	Parallel, independent	Sequential, each fixes the last
Mainly reduces	**Variance (the wobble)**	**Bias (the systematic miss)**
Key knobs	n_estimators, max_features	learning_rate, n_estimators, depth
Temperament	Forgiving, hard to overfit	Sharper — needs watching
Reach for it when	You want a robust baseline	You need the last drop of accuracy

Start with a forest for a sane baseline, then let boosting chase the residual accuracy.

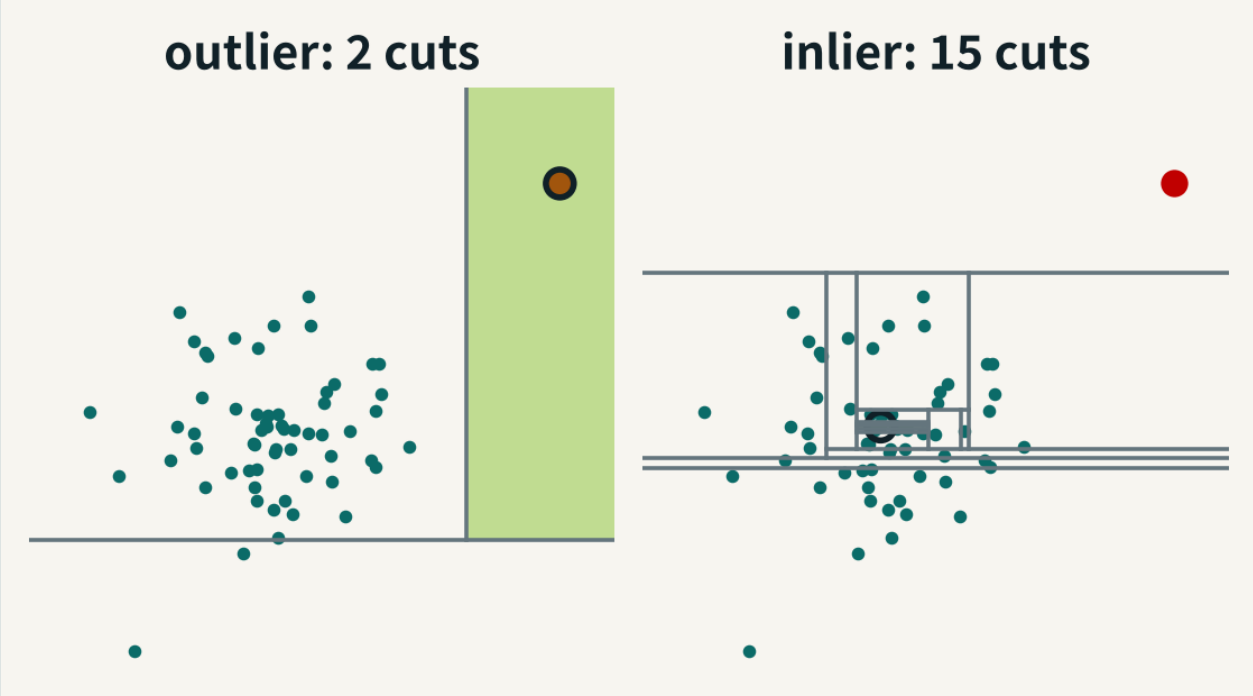
Isolation Forest flags points that few random cuts can isolate

Random axis cuts until the circled point sits alone — anomaly detector #3, no labels needed

Anomaly score (h: cuts needed)

$$s(x, n) = 2^{-\mathbb{E}[h(x)] \,/\, c(n)}$$

Few cuts → score near 1 → odd
Many cuts → score below 0.5 → normal



No distances, no labels, just cuts — fast enough for millions of transactions.

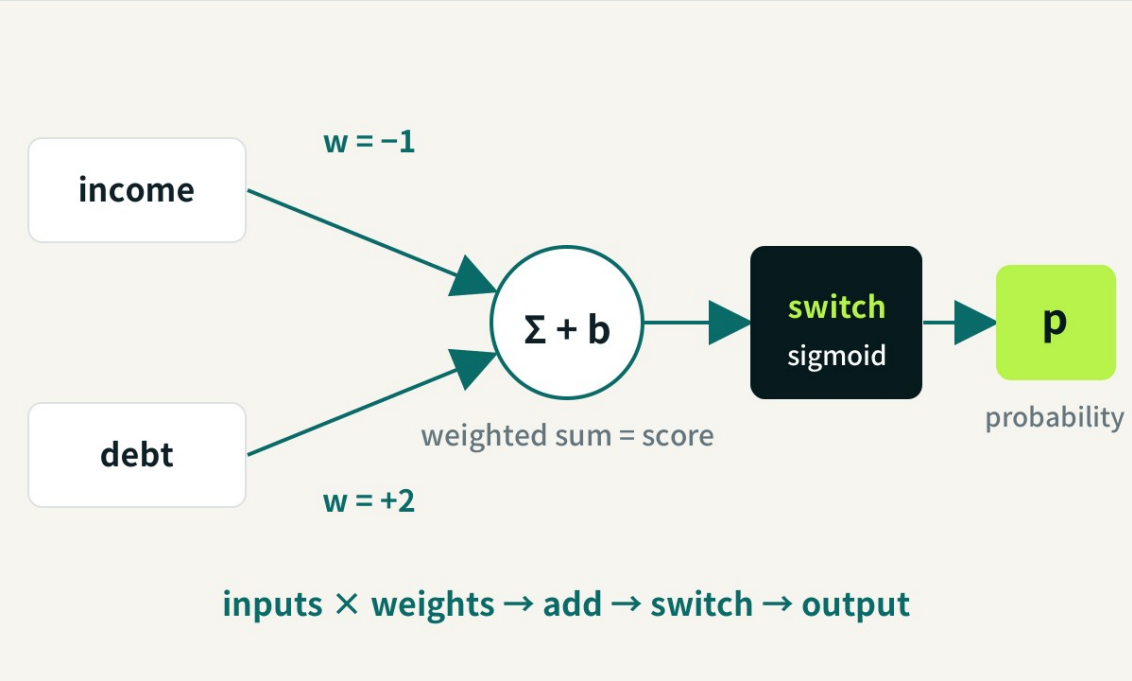
02

Neural Networks

Week 8 · the neuron, depth, learning by gradient descent, and the autoencoder

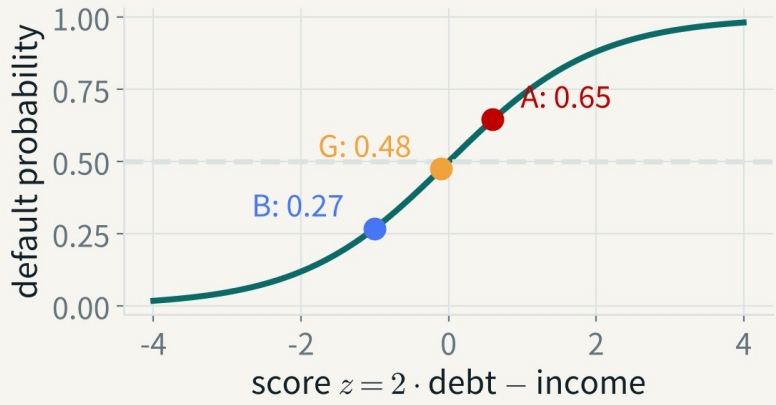
One neuron with a sigmoid is logistic regression

Credit-score neuron: weights (−1, +2) on (income, debt), bias 0



Neuron: weighted sum, then a switch

$$\hat{y} = f\left(\mathbf{w}^\top \mathbf{x} + b\right)$$



Applicant A scores +0.6 → 65% default risk; the power comes from stacking many neurons.

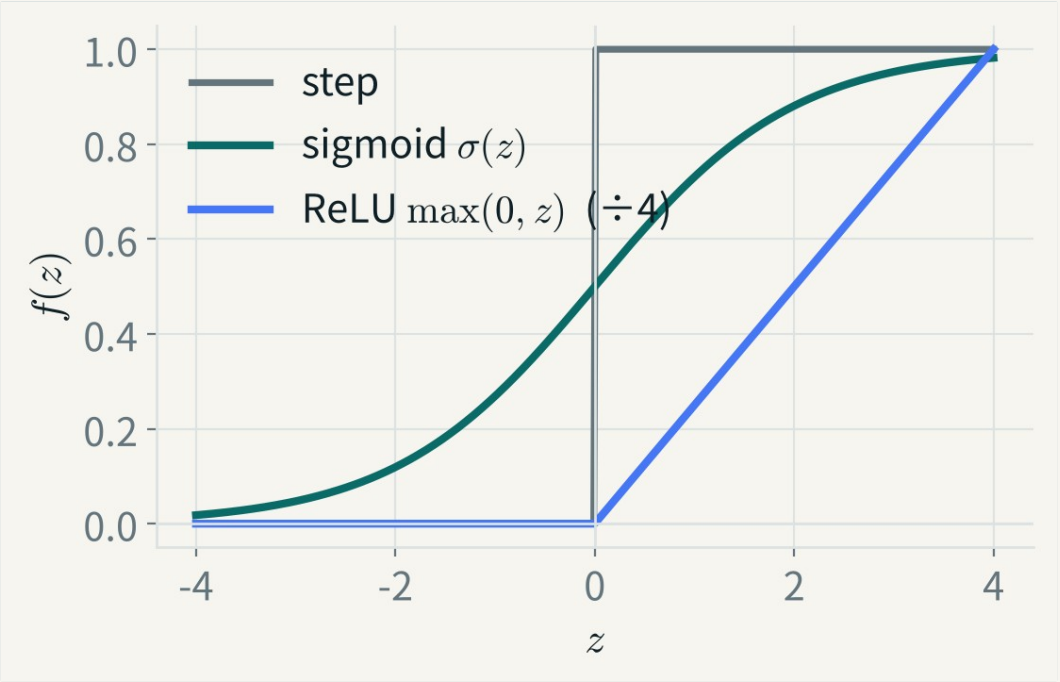
Non-linear activations are what make depth useful

Step, sigmoid and ReLU on the same input range

WHY THE SWITCH

- ReLU passes positives, flattens negatives
- Sigmoid squashes to a probability

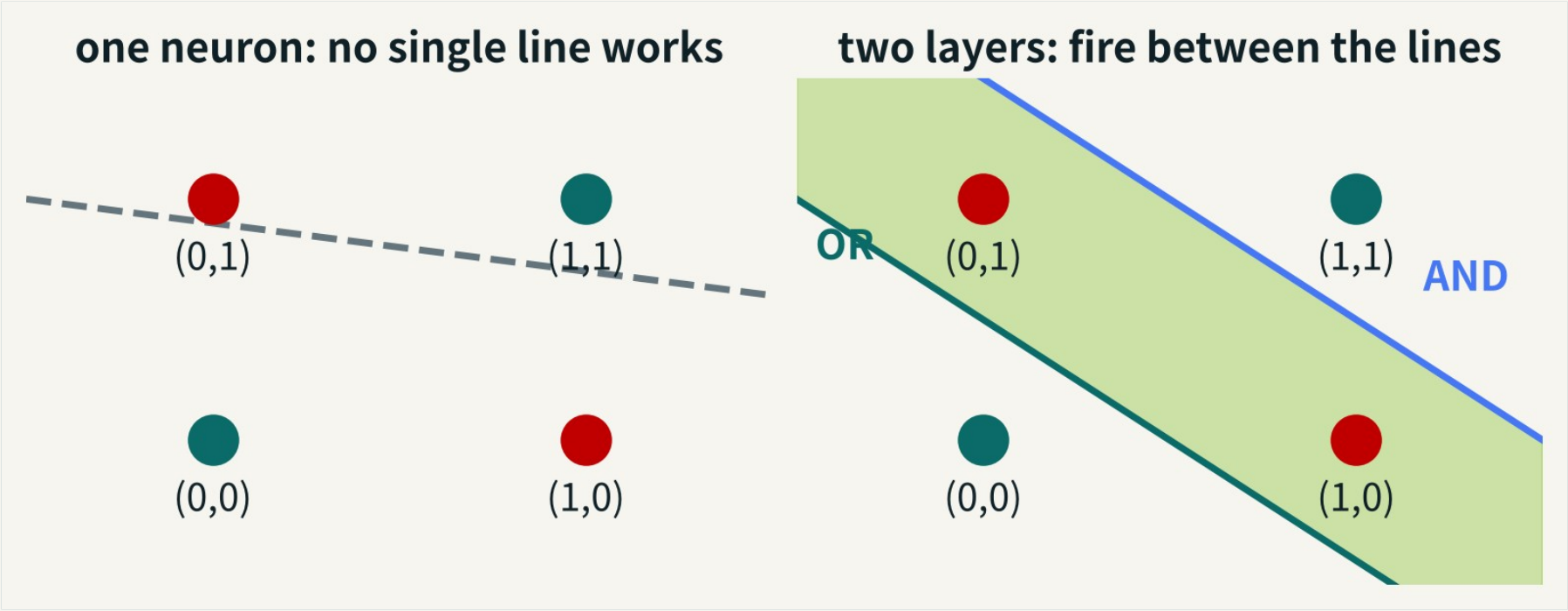
No switch: ten layers = one line



Each layer bends the picture; stacked bends draw rich boundaries.

Two layers solve XOR, which no single neuron can

Fire on exactly one clue — red = fire. Layer 1 draws the OR and AND lines, layer 2 keeps the band between



One neuron draws one line; depth adds bends — the 1969 complaint, answered.

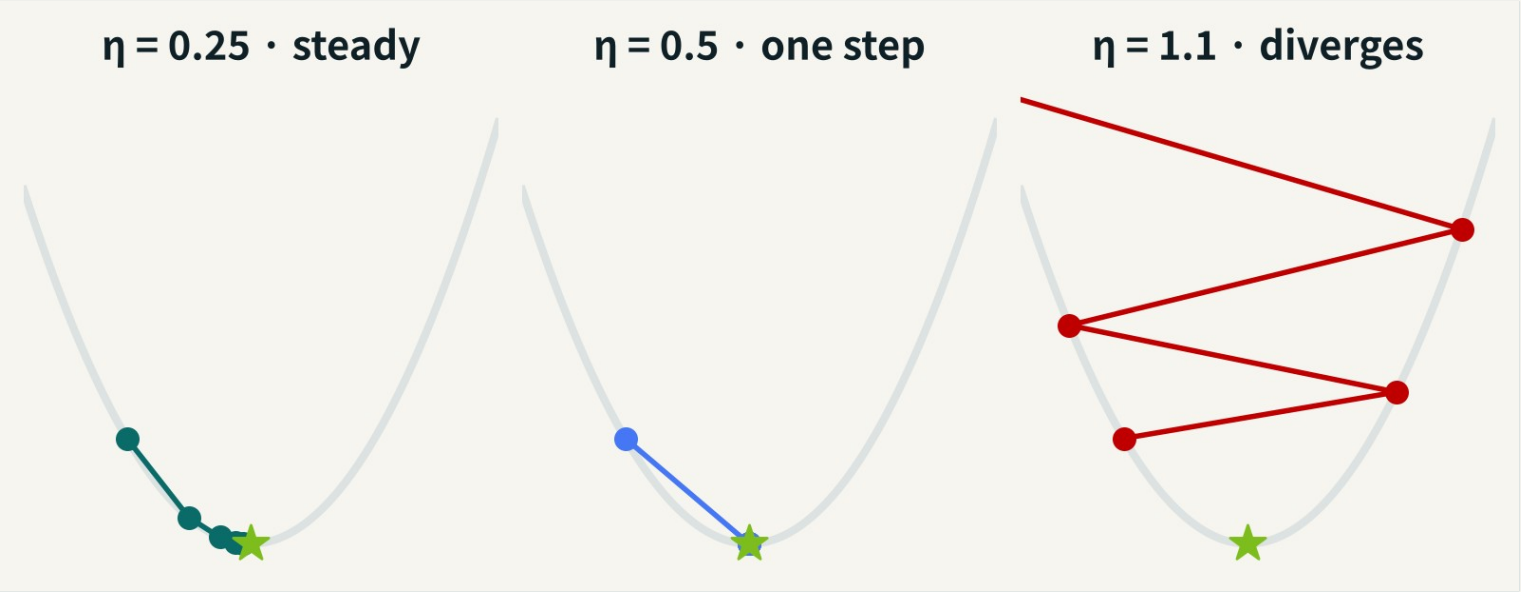
Gradient descent walks downhill by a learning-rate step

Error (w − 3) squared, starting at w = 0 — the star is the valley

The learning step

$$w \leftarrow w - \eta \frac{\partial E}{\partial w}$$

Feel the slope, step downhill, repeat



Small step: steady descent. Just right: one jump. Too big: overshoot and bounce away.

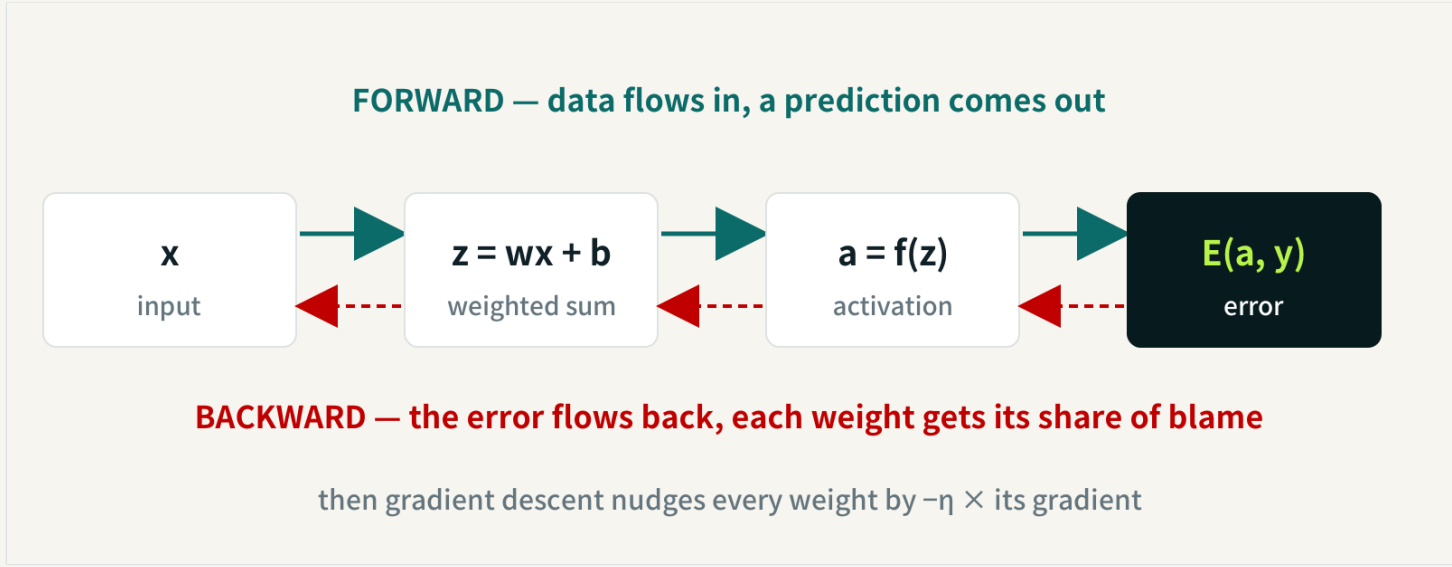
If training loss bounces and explodes, shrink the learning rate.

Backpropagation passes the blame backward with the chain rule

Forward pass makes a prediction; backward pass gives every weight its gradient

Chain rule for one weight

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$



Backprop made many-layer training possible; libraries compute it for you.

Big data + GPUs turned it into today's deep learning.

Deep nets need plenty of data; thin noisy tables favor simpler models

Your data looks like	Reach for	Why
Small, noisy tabular file	**XGBoost or linear**	**Sample-efficient, robust, explainable**
Text: news, filings, chats	**Neural nets → LLMs**	**Learned language representations**
Images: documents, charts	**Deep nets (CNNs)**	**Layers build edges → shapes → objects**
Rich 'normal' to fence	**Autoencoder (#4)**	**Learns the non-linear essence of normal**

Most finance data is a thin noisy table — there, XGBoost often still wins.

An autoencoder flags what it cannot rebuild

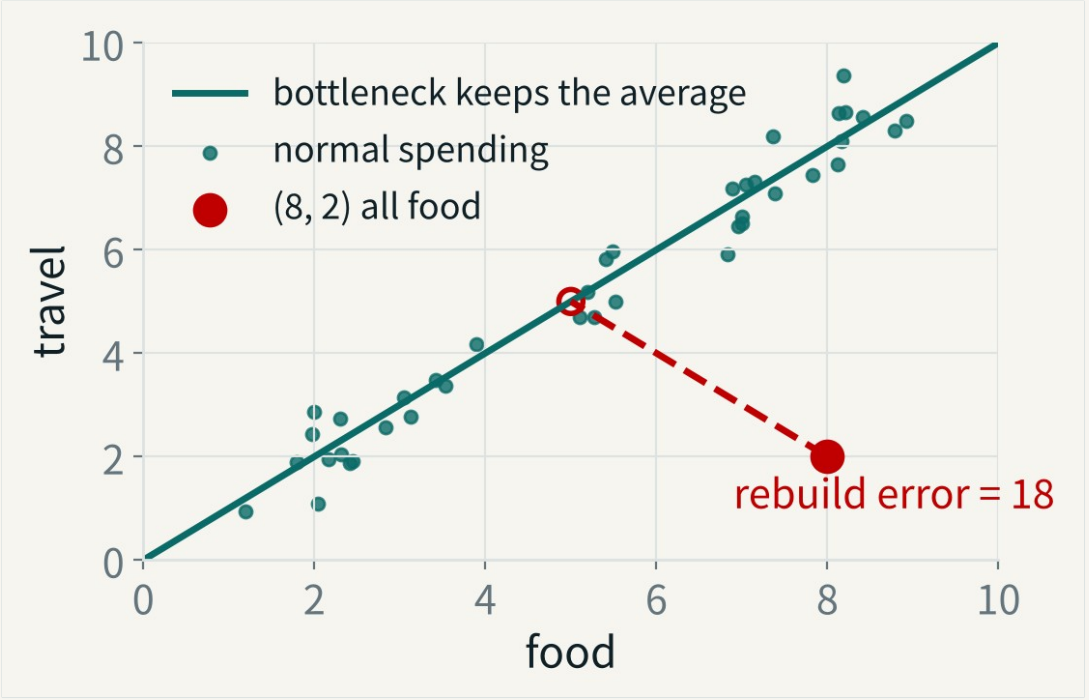
Trained on normal data only — anomaly detector #4; normal spending sits on the diagonal

Rebuild error

$$e(\mathbf{x}) = \left\| \mathbf{x} - g\Big(f(\mathbf{x})\Big) \right\|^2$$

Squeeze → rebuild → compare

Big error = anomaly



Bottleneck too wide = photocopier: anomalies rebuild perfectly and the detector goes blind.

Too narrow = nothing fits. Squeeze until normal just fits.

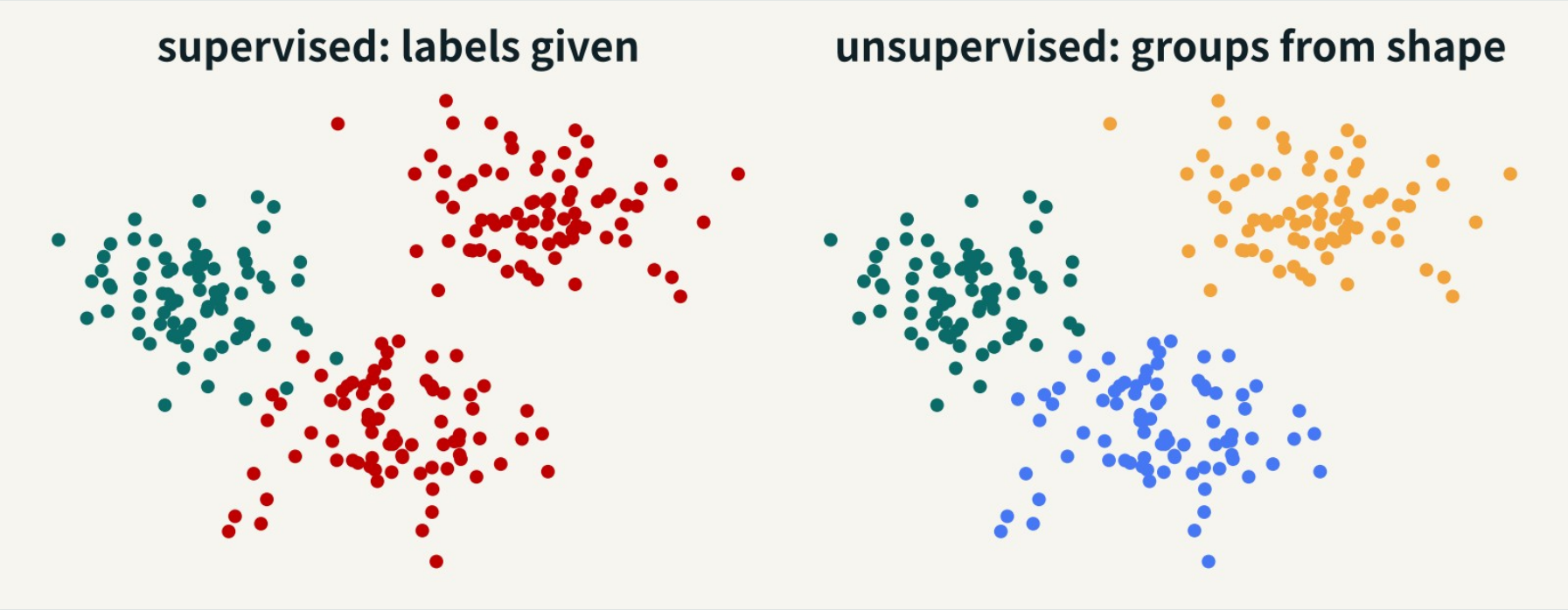
03

Unsupervised Learning

Week 9 · grouping, compressing and cleaning noise — no answer key

Without labels, the goal shifts from prediction to structure

Same kind of data: on the left the answers are given, on the right the algorithm finds the groups



No F1 here: judge whether the structure makes sense and helps.

K-Means alternates assign and update until nothing moves

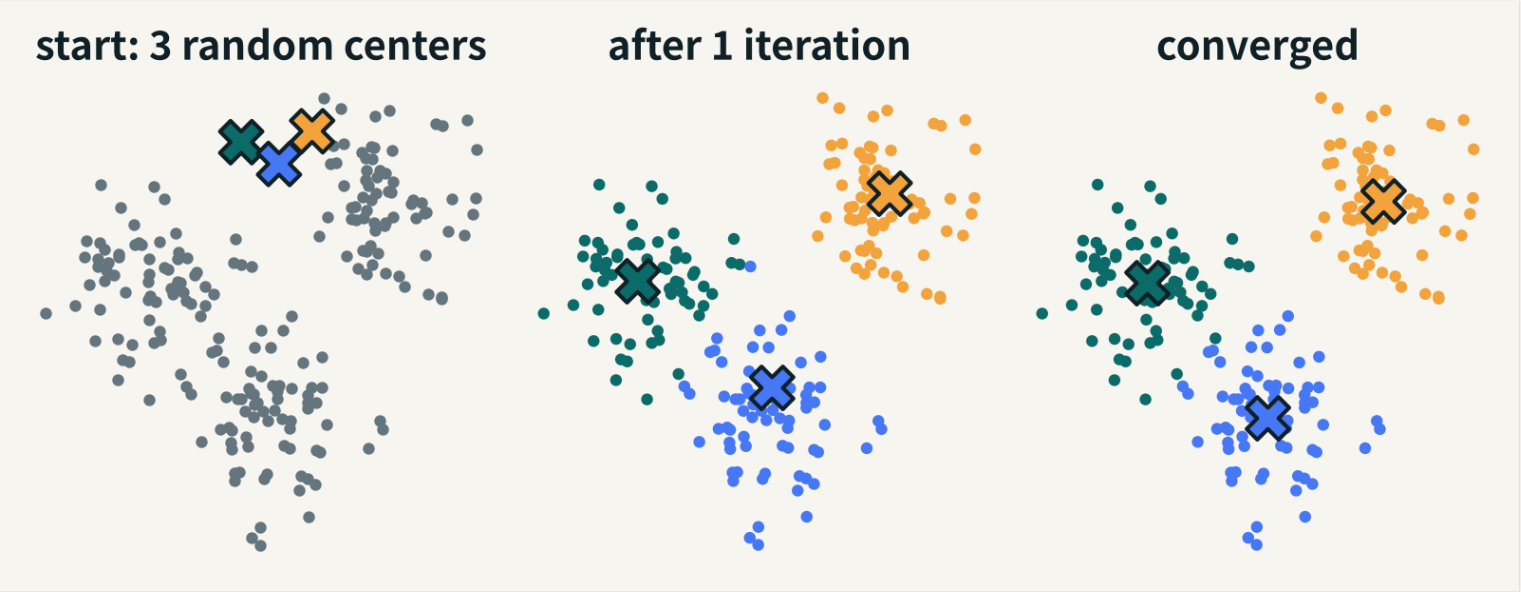
Assign each point to its nearest center, move each center to its group's mean, repeat

Objective

$$J = \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

Keep every point close to its own center

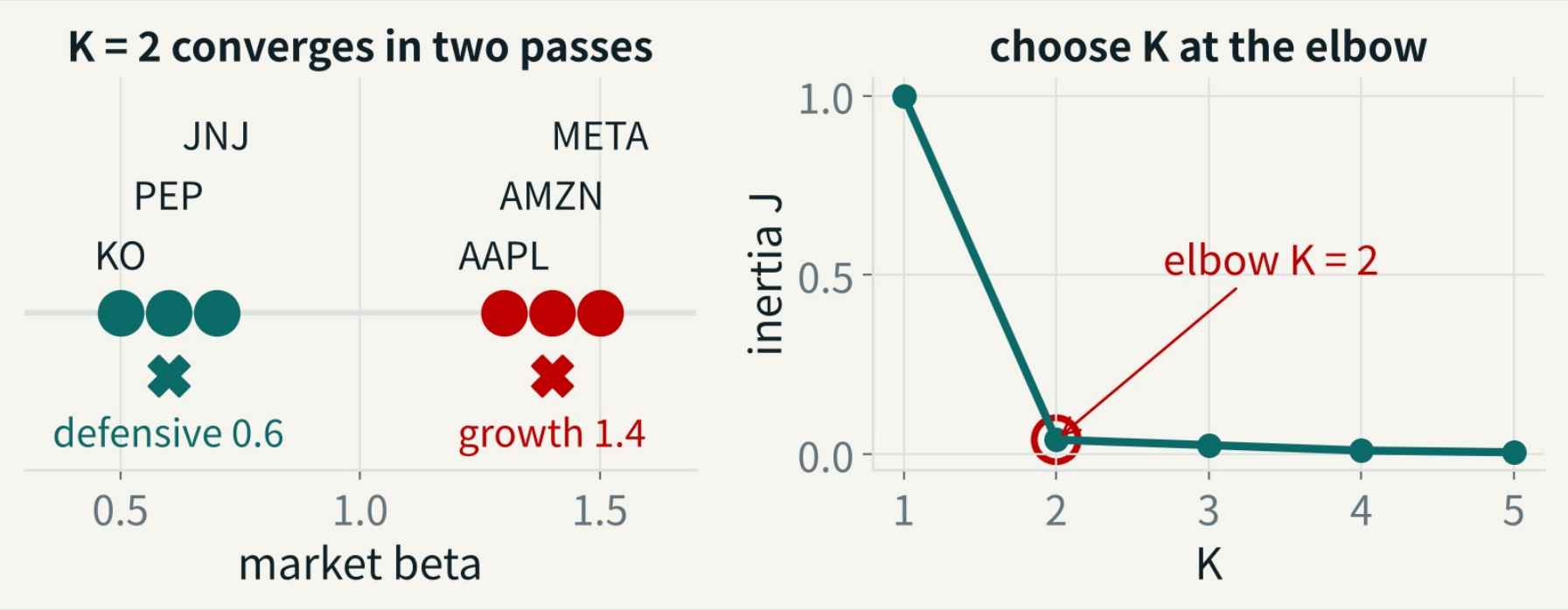
You must pick K



Each pass can only lower the total distance — so the centers settle.

You choose K — the elbow tells you when to stop

Six stocks on market beta, K = 2, start centers 0.5 and 1.5



Inertia falls 1.00 → 0.04 at K = 2, then flattens: defensive vs growth, found without labels.

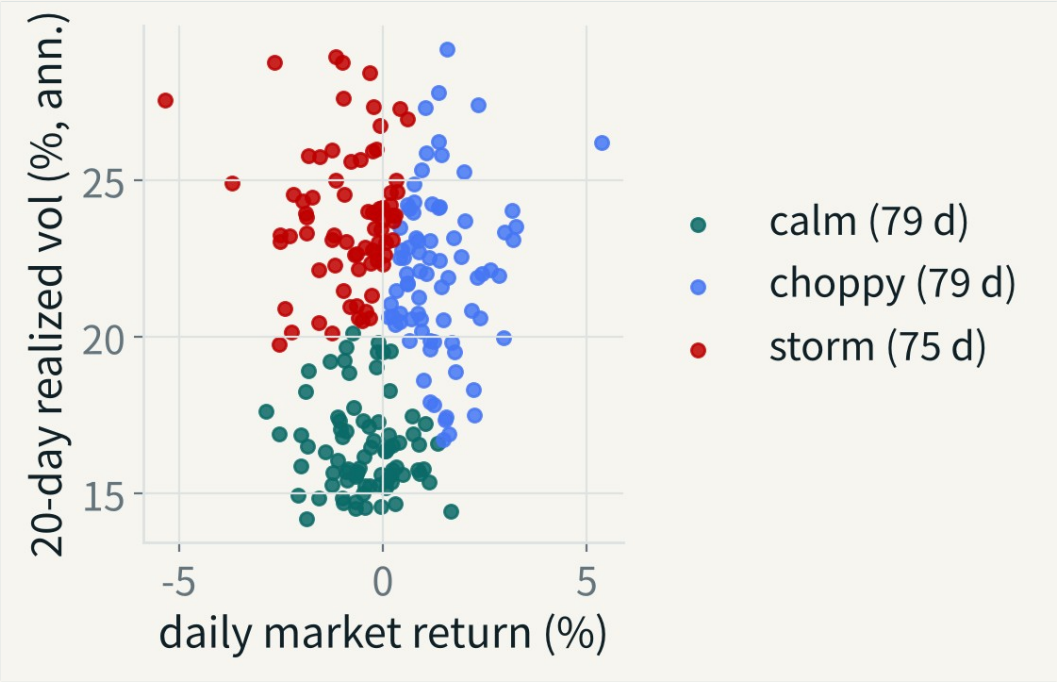
Clustering days instead of stocks reveals market regimes

K-Means, K = 3, on daily (market return, 20-day realized vol) — simulated course panel

THE RECIPE

- One row per day
- Nobody defines a crisis
- Shrink size in storms

Same algorithm, new axis



Models trained on calm days fail in storms — regime clusters say when to switch.

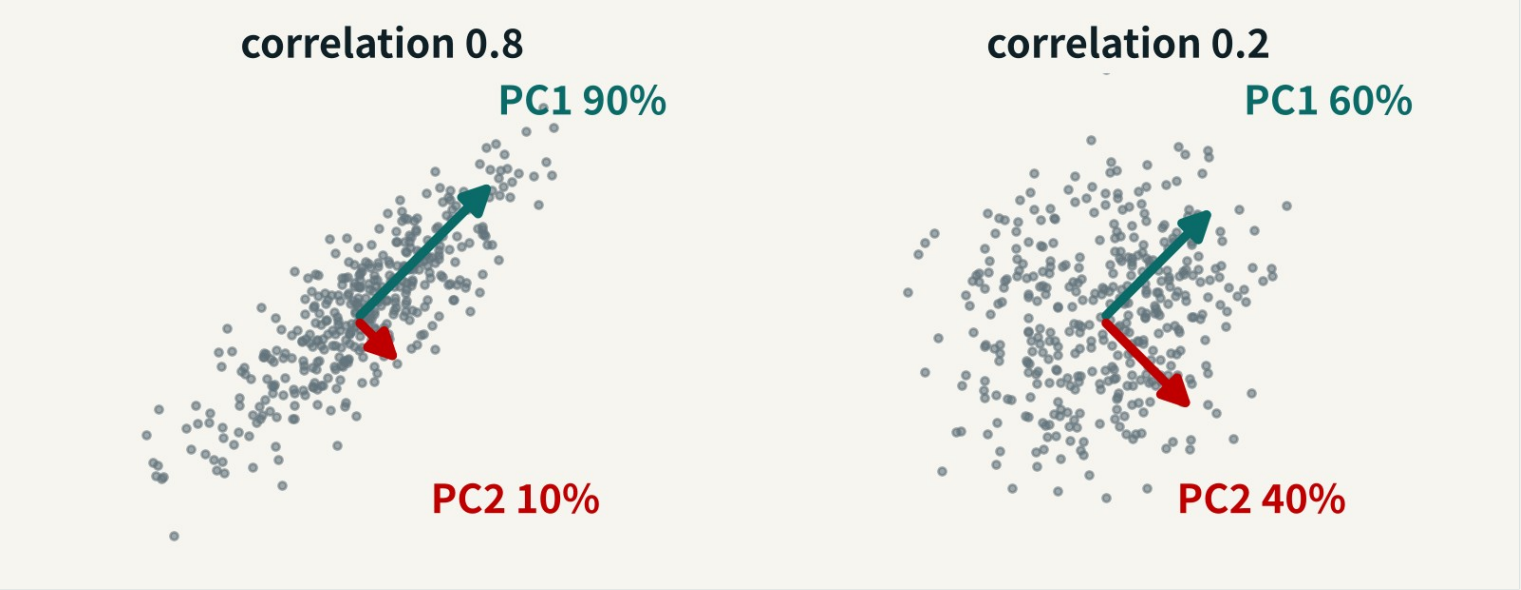
PCA keeps the directions with the largest spread

Two stocks: the stronger the correlation, the more one direction carries

Directions and their share

$$\Sigma v_k = \lambda_k v_k, \qquad \text{share}_k = \frac{\lambda_k}{\sum_j \lambda_j}$$

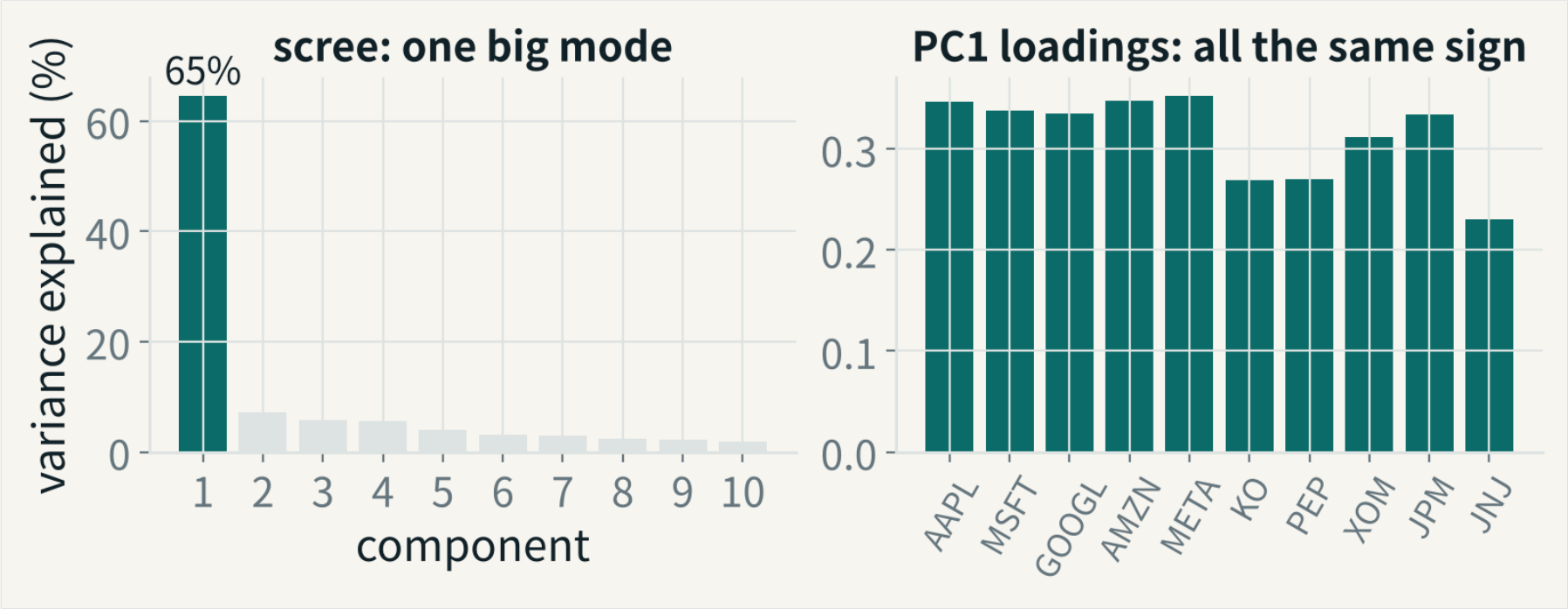
Big eigenvalue =
lots of spread
Keep the top few



Correlation 0.8 → PC1 holds 90%; at 0.2 only 60%. Compression pays when features co-move.

On a stock panel, PC1 is the whole market moving together

Ten-stock simulated course panel, 252 days

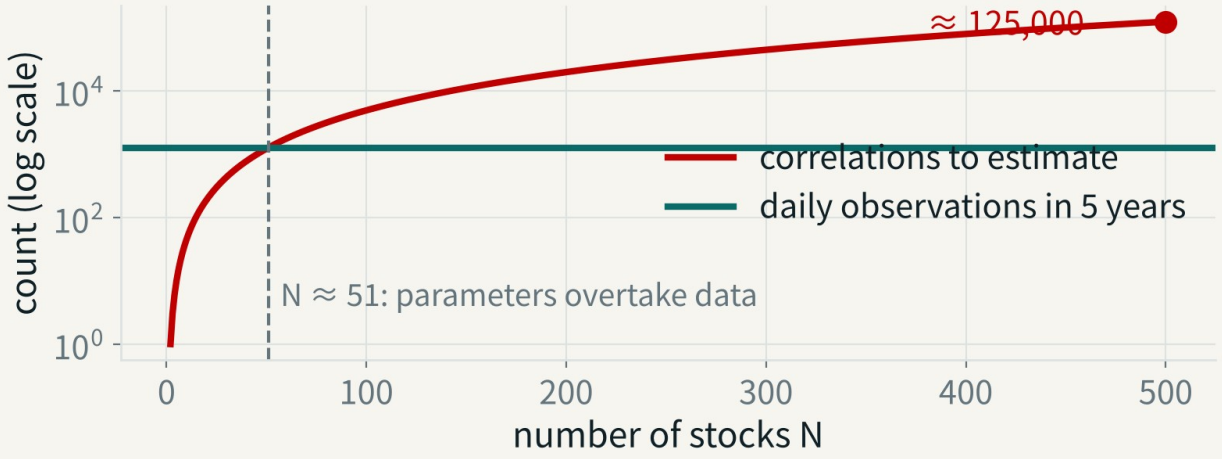


PC1 explains 65%, every loading the same sign; the tail is noise.

With 500 stocks, each correlation rests on almost no evidence

Correlations to estimate vs five years of daily observations

Stocks	Correlation pairs	Trading days (5 yrs)	Days per pair
500	≈ 125,000	1,260	≈ 0.01



Most of a raw correlation matrix is statistical mirage — and optimizers chase it.

Eigenvalues below the Marchenko–Pastur edge are noise

Correlation eigenvalues of the 10-stock simulated course panel, T = 252 days

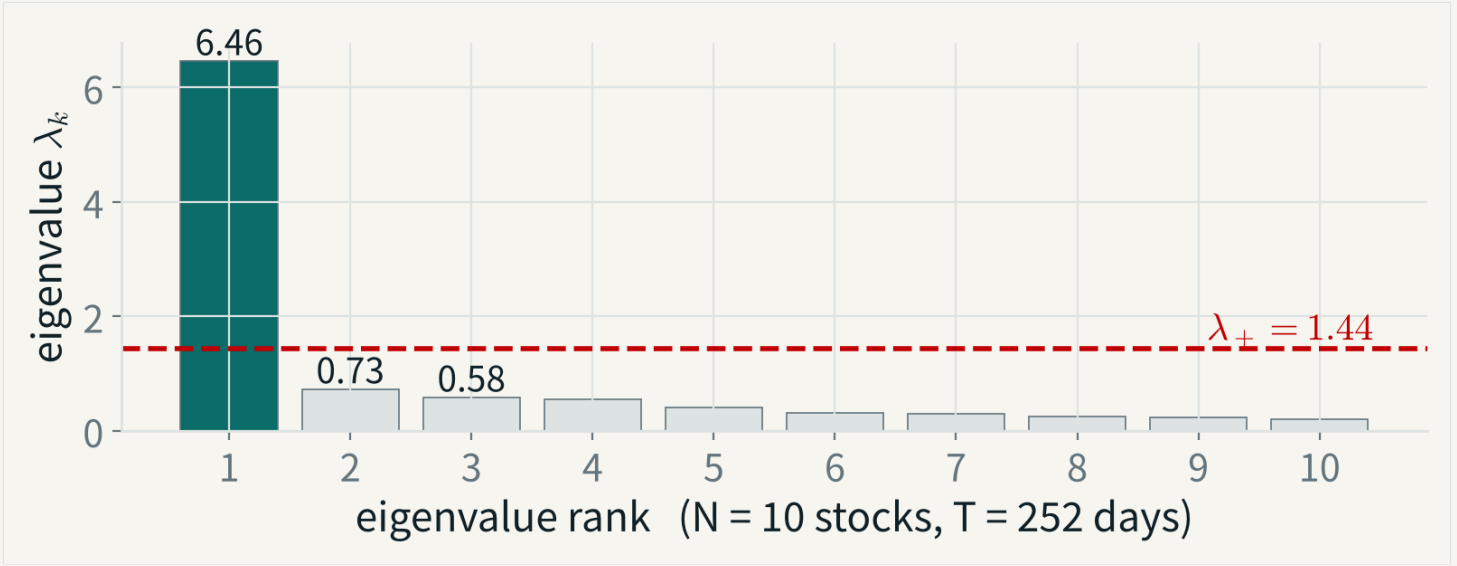
Noise band (N stocks, T days)

$$\lambda_{\pm} = \left(1 \pm \sqrt{N/T}\right)^2$$

SAME EIGENVALUE 2.5

Course panel 1.44 → signal

Big fund 2.66 → noise

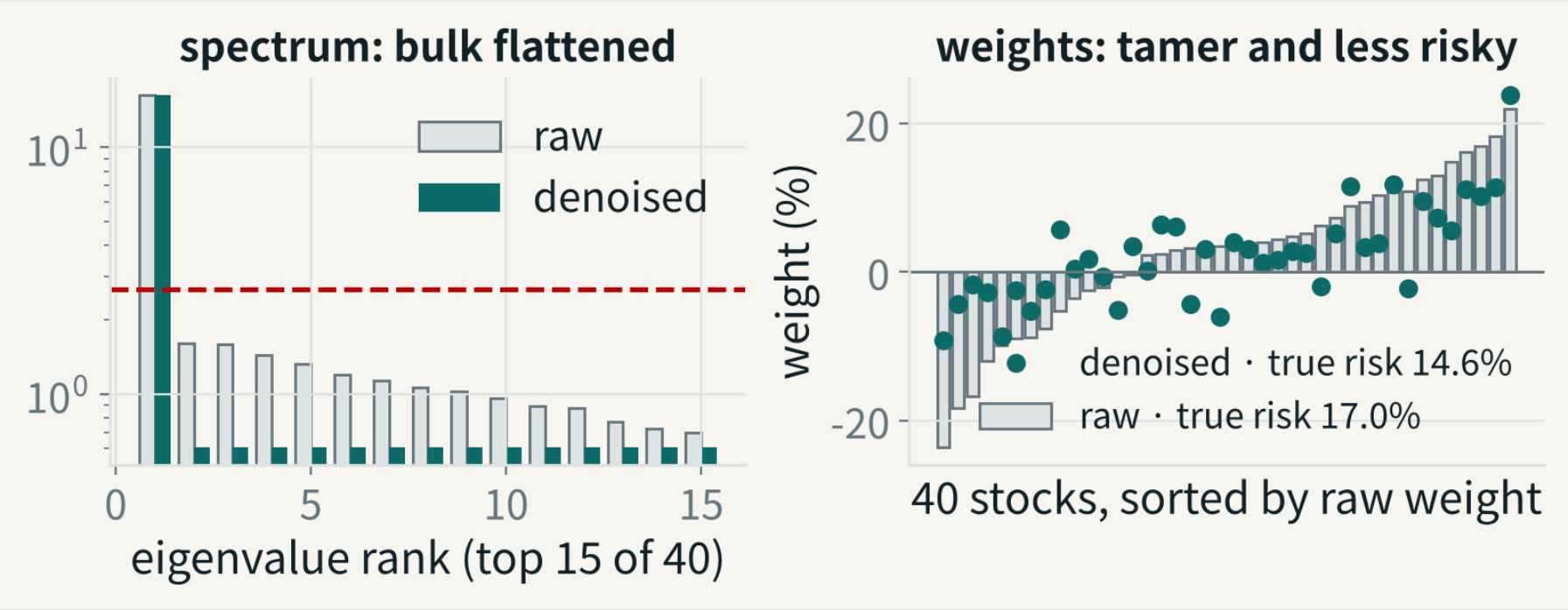


Only 1 of 10 eigenvalues (6.46) clears the edge of 1.44 — the market mode.

More names on the same history raise the edge: yesterday's signal drowns in the bulk.

Denoise the spectrum before the optimizer sees it

Simulated 40 stocks × 100 days: diagonalize → flatten eigenvalues below the edge → rebuild → optimize



Saner weights, lower true risk (17.0% → 14.6%) — feed this to HRP / NCO in Week 11.

04

Cheat Sheet

Weeks 7–9 · core formulas, dials, detectors and the tool picker

Core formulas, Weeks 7–8

Gini impurity (W7)

$$G = 1 - \sum_k p_k^2$$

Variance of a forest (W7)

$$\mathrm{Var}\left[\hat{f}_{\mathrm{bag}}\right] = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2$$

Boosting update (W7)

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$$

Neuron (W8)

$$\hat{y} = f\left(\mathbf{w}^\top \mathbf{x} + b\right)$$

Gradient-descent step (W8)

$$w \;\leftarrow\; w - \eta\,\frac{\partial E}{\partial w}$$

Autoencoder rebuild error (W8)

$$e(\mathbf{x}) = \left\| \mathbf{x} - g\left(f(\mathbf{x})\right) \right\|^2$$

Pure splits · diverse trees · small corrections · careful steps

Core formulas, Week 9

K-Means objective

$$J = \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

PCA

$$\Sigma v_k = \lambda_k v_k, \qquad \text{share}_k = \frac{\lambda_k}{\sum_j \lambda_j}$$

Marchenko–Pastur noise edge

$$\lambda_{\pm} = \left(1 \pm \sqrt{N/T}\right)^2$$

Denoising rule

$$\tilde{\lambda}_k = \bar{\lambda}_{\text{noise}} \quad \text{if } \lambda_k \leq \lambda_+$$

Elbow for K · top eigenvalues for PCA · MP edge before you optimize

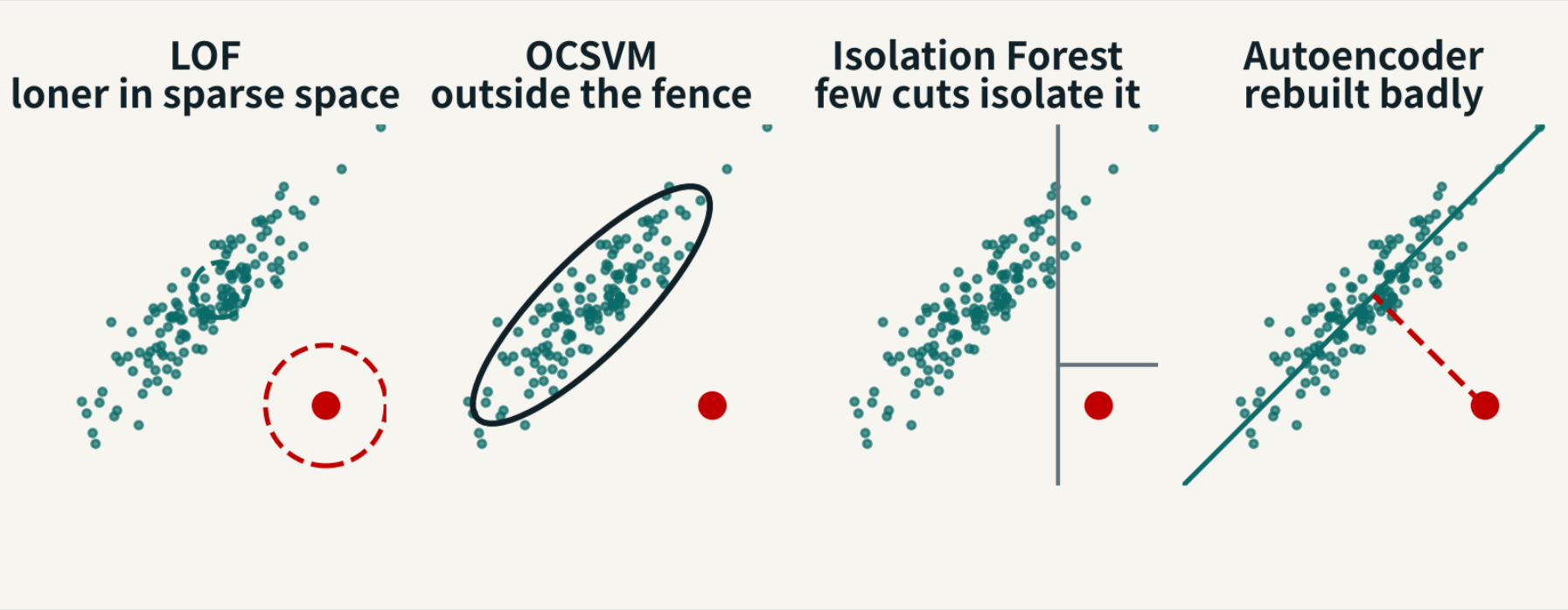
One U-curve, many costumes: flexibility must be paid for with evidence

Dial	Week	Too small	Too large
max_depth (tree)	7	Misses real structure	One leaf per loan
Learning rate	7–8	Training crawls	Overshoots, diverges
Bottleneck size	8	Constant alarms	Photocopier, detector blind
K (clusters)	9	Unlike things lumped	Noise gets its own club
Components kept	9	Signal thrown away	Noise kept

Sweep the dial on validation — or read the elbow when there are no labels.

Four anomaly detectors see 'odd' in four different ways

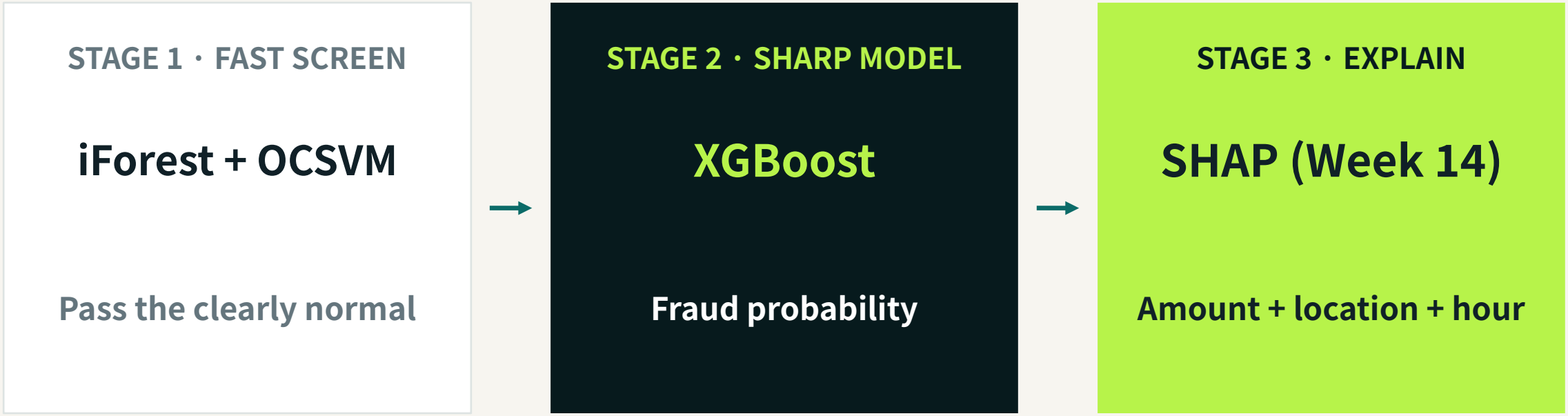
Same data, same red point — LOF (W5) · OCSVM (W6) · Isolation Forest (W7) · Autoencoder (W8)



No single detector is best — banks run several so each covers the others' blind spots.

A production fraud system stacks screen, score and explain

Every tool from Weeks 7–8 has a place in the pipeline



Screen fast → score sharply → explain clearly.

Match the symptom to the tool

Symptom	Tool
A regulator must trace one rejection step by step	A single shallow tree
You need a robust baseline fast	Random Forest
Ten million unlabeled rows, anomalies by lunch	Isolation Forest
Training loss bounces and explodes	Lower the learning rate
The optimizer outputs wild extreme weights	Denoise the covariance first

Diagnose first, then reach for the tool.

ONE TREE MEMORIZES.

A DIVERSE TEAM GENERALIZES.

CLEAN THE STRUCTURE BEFORE YOU OPTIMIZE.

Trees · Networks · Structure
Weeks 7–9